

DC1 Completely Lost: Reconstruct KRaft Control Plane from Surviving Minority C201/C202 or Rebuild a New Quorum

Executive summary and operating boundary

This runbook addresses the following disaster:

Original dynamic KRaft controller quorum

DC1

C101 node.id=101 voter
C102 node.id=102 voter
C103 node.id=103 voter

DC2

C201 node.id=201 voter
C202 node.id=202 voter

CurrentVoters = 5

Quorum majority = 3

After the incident:

DC1

C101 LOST
C102 LOST
C103 LOST

DC2

C201 SURVIVES
C202 SURVIVES

Surviving voters = 2 / 5

Required majority = 3 / 5

=> KRaft metadata quorum is unavailable.

Apache Kafka's KRaft documentation states that a majority of controllers must remain alive for availability; a five-controller quorum tolerates two controller failures, not three.

`controller.quorum.bootstrap.servers` only provides addresses used to discover the active quorum—it does not redefine the set of voters. ¹

For a KIP-853 dynamic quorum, membership is represented by `VotersRecord` state in the replicated metadata log and snapshots. Each voter is identified not merely by `node.id`, but by the combination of replica ID and `directory.id`; the voter record also carries controller endpoints. With `kraft.version=1`, `CurrentVoters` is removed from `quorum-state` because membership is

stored in the snapshot/log instead. ² This matches the storage model already established for this environment. filecite turn0file0

The recovery has two fundamentally different branches:

Branch	Available evidence	Recovery class	RPO interpretation
A — preferred	C201 + C202 + an intact, sufficiently current storage image of one original voter C101/C102/C103	Restore an original 3/5 Raft majority, let KRaft recover normally, then reconfigure membership	RPO=0-capable for KRaft metadata if the recovered voter state is genuinely intact/ current and Raft successfully reconstructs the committed log
B — minority only	Only C201 + C202 remain	Forensic recovery followed by a new quorum / new cluster, or support-assisted offline salvage	RPO=0 cannot be proven from C201/C202 alone

The reason for the Branch B limitation is fundamental. The metadata high watermark is calculated from the majority of voters. Immediately before DC1 disappeared, a metadata record could have been present and committed on C101+C102+C103—three of five—without yet reaching C201 or C202. Therefore equality between C201 and C202, or even identical local log end offsets, does not prove that they contain the last committed metadata state. This is an inference directly from KRaft's majority/high-watermark semantics. ³

For example:

Immediately before DC1 failure:

C101

C102

C103

C201

C202

LEO

10503

10503

10503

10500

10500

Majority = C101+C102+C103

Therefore offsets 10500..10502 may already have been committed even though neither surviving DC2 controller has them.

Conversely, C201 may have an LEO larger than C202 because it received an uncommitted tail:

Known HWM before failure = 10500

C201 LEO = 10505

C202 LEO = 10500

The presence of offsets `10500..10504` on C201 does not prove that those records were committed. **Largest LEO must never be used as the rule for selecting an authoritative recovery seed.** KIP-853 explicitly distinguishes log replication state from majority-derived high watermark semantics. ⁴

Critical safety rule: there is no documented Apache Kafka 4.3 operator command whose semantics are “force these two surviving members of the old five-member quorum to become the authoritative quorum while retaining their existing metadata log.” `add-controller` and `remove-controller` operate through the active leader; `--standalone` bootstraps newly formatted quorum storage rather than promoting an existing minority log. ⁵

This report therefore deliberately does **not** invent an in-place minority promotion procedure.

For current deployments, the best target within the requested range is Kafka **4.3.1**, the supported 4.3 bug-fix release listed by Apache as of September 2, 2026. KIP-853 dynamic quorum support was delivered earlier, while Kafka 4.1 added the documented static-to-dynamic quorum upgrade path and `kraft.version=1` operational workflow. For an actual incident, however, run the **same Kafka binary and patch level that produced the on-disk metadata** until recovery is complete; do not combine DR with a Kafka or metadata-feature upgrade. ⁶

Primary references for the operator should be kept beside the runbook:

- [Apache Kafka 4.3 — KRaft Operations](#) ¹
- [KIP-853 — KRaft Controller Membership Changes](#) ⁷
- [KIP-630 — Kafka Raft Snapshot](#) ⁸
- [Apache Kafka 4.3 — Hardware and OS](#) ⁹
- [Apache Kafka 4.3 — Basic Operations](#) ¹⁰
- [Apache Kafka 4.3 — Distribution / consumer offsets](#) ¹¹
- [Apache Kafka 4.3 — Broker Configs](#) ¹²

Prerequisites, assumptions, and first-page incident checklist

The exact Kafka version, packaging layout, controller service name, authentication mechanism, broker topology, broker data paths, topic replication factors, DC2 broker IDs, storage technology, backup technology, TLS/SASL configuration, and whether the servers use isolated or combined broker/controller roles were not specified. The commands below therefore assume GNU/Linux, Bash, systemd, separate controllers, Kafka 4.1–4.3 dynamic KRaft with `kraft.version >= 1`, and `/opt/kafka` as `KAFKA_HOME`. Adapt paths and service names, but **do not alter the recovery semantics**. Apache recommends separate controller and broker roles for critical deployments. ¹

The examples assume:

Controller	node.id
-----	-----
c101.dc1.example.com	101
c102.dc1.example.com	102
c103.dc1.example.com	103
c201.dc2.example.com	201
c202.dc2.example.com	202

CONTROLLER port	9093
metadata.log.dir	/var/lib/kafka/kraft-metadata
controller systemd unit	kafka-controller
Kafka installation	/opt/kafka
Admin-client security properties	/etc/kafka/quorum-admin.properties

Do not assume `/var/lib/kafka/kraft-metadata` from this document during a real incident. Establish it from the deployed controller configuration. Kafka uses `metadata.log.dir` for the metadata log; if it is unset, Kafka's storage configuration determines the effective location, so the actual production configuration must be recorded before copying anything. ¹²

The first-page operator environment:

```
set -o pipefail

export KAFKA_HOME=/opt/kafka
export PATH="$KAFKA_HOME/bin:$PATH"

export CTRL_PORT=9093

export C101=c101.dc1.example.com
export C102=c102.dc1.example.com
export C103=c103.dc1.example.com
export C201=c201.dc2.example.com
export C202=c202.dc2.example.com

export META_DIR=/var/lib/kafka/kraft-metadata
export META_PART="${META_DIR}/__cluster_metadata-0"

export KAFKA_SERVICE=kafka-controller
export ADMIN_CONFIG=/etc/kafka/quorum-admin.properties

export INCIDENT_ID="dc1-loss-$(date -u +%Y%m%dT%H%M%S)"
export CENTRAL_EVIDENCE="/recovery/${INCIDENT_ID}"

mkdir -p "$CENTRAL_EVIDENCE"

printf 'Incident: %s\nUTC start: %s\n' \
"$INCIDENT_ID" "$(date -u --iso-8601=seconds)"
```

Before changing anything, record the installed Kafka version. Apache Kafka's 4.3 quickstart currently uses 4.3.1, but this incident must be recovered with the deployed version rather than opportunistically upgraded. ¹³

```
"$KAFKA_HOME/bin/kafka-topics.sh" --version
```

```
ssh "$C201" "$KAFKA_HOME/bin/kafka-topics.sh --version"
ssh "$C202" "$KAFKA_HOME/bin/kafka-topics.sh --version"
```

Attempt to capture quorum status from each survivor. With 2/5 voters, timeout or inability to locate a leader is expected; **save the failure output because it is incident evidence.** `describe --status` normally reports `ClusterId`, `LeaderId`, `LeaderEpoch`, `HighWatermark`, follower lag and the current voter/observer identities. ⁵

```
set +e

timeout 20s kafka-metadata-quorum.sh \
  --command-config "$ADMIN_CONFIG" \
  --bootstrap-controller "${C201}:${CTRL_PORT}" \
  describe --status \
  2>&1 | tee "$CENTRAL_EVIDENCE/pre-freeze-status-via-c201.txt"

printf 'exit_code=%s\n' "${PIPESTATUS[0]}" \
  | tee -a "$CENTRAL_EVIDENCE/pre-freeze-status-via-c201.txt"

timeout 20s kafka-metadata-quorum.sh \
  --command-config "$ADMIN_CONFIG" \
  --bootstrap-controller "${C202}:${CTRL_PORT}" \
  describe --status \
  2>&1 | tee "$CENTRAL_EVIDENCE/pre-freeze-status-via-c202.txt"

printf 'exit_code=%s\n' "${PIPESTATUS[0]}" \
  | tee -a "$CENTRAL_EVIDENCE/pre-freeze-status-via-c202.txt"

set -e
```

Also try replication state:

```
set +e

timeout 20s kafka-metadata-quorum.sh \
  --command-config "$ADMIN_CONFIG" \
  --bootstrap-controller "${C201}:${CTRL_PORT}" \
  describe --replication \
  2>&1 | tee "$CENTRAL_EVIDENCE/pre-freeze-replication.txt"

set -e
```

When a quorum is healthy, current Kafka output includes the replica ID/directory ID, log end offset, lag and fetch/caught-up timestamps. Those fields answer different questions: `LogEndOffset` tells you what the local replica has appended; `Lag` compares it with the active leader; timestamps indicate recent catch-up; they do **not** independently establish a commit boundary. ¹⁴

Search monitoring, central logs and ticket history for the **last successful pre-incident** `describe --status`. A recorded `HighWatermark` from shortly before the DC1 loss is disproportionately valuable because it provides an externally captured committed-prefix boundary. The runtime high watermark is majority-derived and monotonically protected by the leader; local minority LEO values are not a substitute. ⁴

The first irreversible safety checkpoint is:

```
[ ] Producers and metadata-changing automation are fenced.
[ ] C101/C102/C103 cannot unexpectedly reappear on the network.
[ ] C201/C202 identities are known.
[ ] Last known healthy quorum status/HWM has been searched for.
[ ] C201/C202 are stopped before forensic copying.
[ ] Controller and broker disks have not been formatted.
[ ] No add-controller/remove-controller attempt has been made.
[ ] No meta.properties or quorum-state file has been edited.
[ ] No original broker data directory has been attached to a new empty quorum.
```

Apache's own failed-metadata-disk guidance is deliberately conservative: a replacement controller should not be formatted and started until a majority has all committed data. The rationale is that letting controllers participate after loss of persistent state can violate Raft's durability assumptions and lose committed records. ¹⁵

Stop C201 and C202:

```
for h in "$C201" "$C202"; do
  echo "=== Stopping controller on $h ==="

  ssh "$h" \
    "sudo systemctl stop '$KAFKA_SERVICE'"

  ssh "$h" \
    "sudo systemctl is-active '$KAFKA_SERVICE' || true"

  ssh "$h" \
    "pgrep -af 'kafka|Kafka' || true"
done
```

Do not format either survivor:

```
FORBIDDEN ON ORIGINAL C201/C202 STORAGE:

kafka-storage.sh format ...
kafka-storage.sh format --standalone ...
kafka-storage.sh format --initial-controllers ...
rm -rf __cluster_metadata-0
```

```
rm meta.properties
rm quorum-state
```

`--ignore-formatted` is not a disaster-recovery bypass. Apache documents it only for a specific replacement situation in combined mode where the metadata directory was lost but other already-formatted log directories remain; it is not permission to overwrite surviving quorum evidence. ¹⁶

Forensic preservation and state interpretation

The forensic objective is not merely “find the latest file.” It is to preserve enough information to answer four separate questions:

```
WHO IS THIS STORAGE?
    meta.properties

WHAT PERSISTENT RAFT/ELECTION STATE DOES IT HAVE?
    quorum-state

WHAT METADATA HISTORY/IMAGE DOES IT HAVE?
    __cluster_metadata-0 log + snapshots

WHO DOES THAT METADATA HISTORY SAY THE VOTERS ARE?
    VotersRecord
```

That separation is essential under KIP-853. ¹⁷

State-source matrix

State source	Typical location	What it tells the operator	Authority in dynamic KRaft	
<code>meta.properties</code>	<code>metadata.log.dir/</code> <code>meta.properties</code>	<code>cluster.id</code> , <code>node.id</code> , <code>directory.id</code> , storage version	Authoritative local storage identity	W p C m c
<code>quorum-state</code>	<code>__cluster_metadata-0/</code> <code>quorum-state</code>	Persistent Raft election/voting state such as leader/vote epoch information	Authoritative for that replica's local persisted Raft state	D u I

State source	Typical location	What it tells the operator	Authority in dynamic KRaft	
<code>VotersRecord</code>	Metadata log and snapshots	Complete voter IDs, directory IDs and controller endpoints	Dynamic membership state as replayed by KRaft	B ra a p c fa
KRaft snapshot	<code><endOffset>-<epoch>.checkpoint</code>	Metadata image at a snapshot boundary, including KRaft control records	Part of replicated metadata state	C a
Metadata log after snapshot	<code>*.log</code>	Ordered changes after the snapshot	Completes the current metadata image	C lo ta e
<code>controller.quorum.bootstrap.servers</code>	properties file	Addresses used to discover quorum/leader	Discovery only	M
Last recorded <code>HighWatermark</code>	Monitoring / healthy <code>describe --status</code>	Committed-prefix boundary at observation time	Extremely strong external recovery evidence	R a o
Broker observer metadata	Broker metadata directory	Metadata state replicated to a broker observer	Valuable forensic evidence	A c 3

Apache's 4.3 `describe --status` output explicitly includes `CurrentVoters` entries with both `id` and `directoryId`, plus endpoints, and also lists `CurrentObservers`. KIP-853 specifies the same identity model in `VotersRecord`; `controller.quorum.bootstrap.servers` is explicitly analogous to client bootstrap discovery rather than membership. ¹⁸

KIP-853 further specifies that `CurrentVoters` is removed from version-1 `quorum-state`: voter state is instead stored in the snapshot and log. This is why copying, editing or reading `quorum-state` as though it were the membership database is incorrect. ¹⁷

KIP-630 establishes snapshot semantics: after loading a snapshot, Kafka applies replicated-log operations from the snapshot end offset onward; snapshot IDs encode end offset and epoch. Therefore the useful mental model is:

```
effective metadata state
=
latest applicable valid snapshot
```

```
+
records after snapshot
```

rather than “find the latest `VotersRecord` text anywhere.” ¹⁹

Create local forensic copies on each survivor. The following assumes enough capacity exists under `/recovery`. The controllers must already be stopped.

```
for h in "$C201" "$C202"; do
  ssh "$h" "sudo mkdir -p '/recovery/${INCIDENT_ID}'"

  ssh "$h" "
    sudo rsync \
      -aHAX \
      --numeric-ids \
      '${META_DIR}/' \
      '/recovery/${INCIDENT_ID}/kraft-metadata/'
  "
done
```

Create deterministic SHA-256 manifests over the copied files:

```
for h in "$C201" "$C202"; do
  ssh "$h" "
    set -euo pipefail

    COPY='/recovery/${INCIDENT_ID}/kraft-metadata'
    OUT='/recovery/${INCIDENT_ID}/kraft-metadata.sha256'

    (
      cd "\"$COPY\"
      find . -type f -print0 \
        | LC_ALL=C sort -z \
        | xargs -0 sha256sum
    ) > "\"$OUT\"

    sha256sum "\"$OUT\"
  "
done
```

Create immutable transfer archives:

```
for h in "$C201" "$C202"; do
  ssh "$h" "
    set -euo pipefail

    cd '/recovery/${INCIDENT_ID}'
```

```

sudo tar \
  --xattrs \
  --acls \
  --numeric-owner \
  -cpf '${INCIDENT_ID}-${h}-kraft-metadata.tar' \
  kraft-metadata

sudo sha256sum \
  '${INCIDENT_ID}-${h}-kraft-metadata.tar' \
  > '${INCIDENT_ID}-${h}-kraft-metadata.tar.sha256'
"
done

```

Pull the archives to an independent evidence host:

```

mkdir -p "$CENTRAL_EVIDENCE/C201" "$CENTRAL_EVIDENCE/C202"

rsync -a \
  "${C201}:/recovery/${INCIDENT_ID}/" \
  "$CENTRAL_EVIDENCE/C201/"

rsync -a \
  "${C202}:/recovery/${INCIDENT_ID}/" \
  "$CENTRAL_EVIDENCE/C202/"

```

Verify after transfer:

```

(
  cd "$CENTRAL_EVIDENCE/C201"
  sha256sum -c "${INCIDENT_ID}-${C201}-kraft-metadata.tar.sha256"
)

(
  cd "$CENTRAL_EVIDENCE/C202"
  sha256sum -c "${INCIDENT_ID}-${C202}-kraft-metadata.tar.sha256"
)

```

Preserve OS and service evidence too:

```

for h in "$C201" "$C202"; do
  mkdir -p "$CENTRAL_EVIDENCE/$h"

  ssh "$h" \
    "date --iso-8601=seconds; timedatectl; uname -a" \
    > "$CENTRAL_EVIDENCE/$h/os.txt" 2>&1

  ssh "$h" \

```

```

    "sudo systemctl status '$KAFKA_SERVICE' --no-pager || true" \
    > "$CENTRAL_EVIDENCE/$h/service-status.txt" 2>&1

ssh "$h" \
    "sudo journalctl -u '$KAFKA_SERVICE' --since '24 hours ago' --no-pager" \
    > "$CENTRAL_EVIDENCE/$h/controller-journal.txt" 2>&1

ssh "$h" \
    "ss -lntp; getent hosts '$C201'; getent hosts '$C202'" \
    > "$CENTRAL_EVIDENCE/$h/network.txt" 2>&1
done

```

Preserve server configuration securely as evidence, because controller listeners, security protocols and storage locations can explain why election or recovery fails. Do not paste unredacted passwords, JAAS entries or private-key material into ordinary incident tickets.

Read identity only from the **copy**:

```

for h in "$C201" "$C202"; do
    echo "==== $h ===="

    grep -E \
        '^(cluster.id|node.id|directory.id|version)= ' \
        "$CENTRAL_EVIDENCE/$h/kraft-metadata/meta.properties"
done

```

Expected conceptually:

```

C201:
cluster.id=q1Sh-9_ISia_zwGINzRvyQ
node.id=201
directory.id=UUID201
version=1

C202:
cluster.id=q1Sh-9_ISia_zwGINzRvyQ
node.id=202
directory.id=UUID202
version=1

```

Checkpoint: the `cluster.id` values must agree. A mismatch is a hard stop because it indicates that the two directories do not represent the same Kafka cluster incarnation. Do not “fix” the mismatch by editing `meta.properties`.

Capture `quorum-state` without manipulating it:

```

for h in "$C201" "$C202"; do
    QS="$CENTRAL_EVIDENCE/$h/kraft-metadata/__cluster_metadata-0/quorum-state"

    stat "$QS" > "$CENTRAL_EVIDENCE/$h/quorum-state.stat"
    sha256sum "$QS" > "$CENTRAL_EVIDENCE/$h/quorum-state.sha256"
done

```

Do **not** copy C201's `quorum-state` onto C202. Do **not** copy `meta.properties` between controllers. Persistent votes and directory identities are per-replica state; Apache's failed-disk analysis specifically explains why loss or reuse of persistent voting state can violate Raft assumptions. ²⁰

Inventory the metadata partition:

```

for h in "$C201" "$C202"; do
    PART="$CENTRAL_EVIDENCE/$h/kraft-metadata/__cluster_metadata-0"

    find "$PART" \
        -maxdepth 1 \
        -type f \
        -printf '%f\t%s\t%TY-%Tm-%TdT%TH:%TM:%TS\n' \
        | LC_ALL=C sort \
        > "$CENTRAL_EVIDENCE/$h/metadata-files.tsv"
done

diff -u \
    "$CENTRAL_EVIDENCE/$C201/metadata-files.tsv" \
    "$CENTRAL_EVIDENCE/$C202/metadata-files.tsv" \
    || true

```

Decode every metadata log segment with the Kafka binary version matching the incident cluster. Apache documents `kafka-dump-log.sh --cluster-metadata-decoder --files` specifically for metadata logs and snapshots. ¹

```

for h in "$C201" "$C202"; do
    PART="$CENTRAL_EVIDENCE/$h/kraft-metadata/__cluster_metadata-0"
    OUT="$CENTRAL_EVIDENCE/$h/decoded"

    mkdir -p "$OUT"

    for f in "$PART"/*.log; do
        [ -e "$f" ] || continue

        base="$(basename "$f")"

        kafka-dump-log.sh \
            --cluster-metadata-decoder \
            --files "$f" \
            > "$OUT/${base}.decoded.txt" 2>&1
    done
done

```

```
done
done
```

Decode snapshot files separately:

```
for h in "$C201" "$C202"; do
    PART="$CENTRAL_EVIDENCE/$h/kraft-metadata/__cluster_metadata-0"
    OUT="$CENTRAL_EVIDENCE/$h/decoded"

    while IFS= read -r -d '' f; do
        base="$(basename "$f")"

        kafka-dump-log.sh \
            --cluster-metadata-decoder \
            --files "$f" \
            > "$OUT/${base}.decoded.txt" 2>&1

    done < <(
        find "$PART" \
            -maxdepth 1 \
            -type f \
            -name '[0-9]*-[0-9]*.checkpoint' \
            -print0
    )
done
```

Search the decoded material:

```
for h in "$C201" "$C202"; do
    echo "==== $h KRaft control history ====="

    grep -RniE \
        'VotersRecord|KRaftVersionRecord' \
        "$CENTRAL_EVIDENCE/$h/decoded" \
        || true
done
```

For broader reconstruction evidence:

```
for h in "$C201" "$C202"; do
    grep -RniE \
        'TopicRecord|PartitionRecord|ConfigRecord|VotersRecord|
        KRaftVersionRecord' \
        "$CENTRAL_EVIDENCE/$h/decoded" \
        > "$CENTRAL_EVIDENCE/$h/important-metadata-records.txt" \
```

```
|| true
done
```

Use `kafka-metadata-shell.sh` against a **real non-bootstrap snapshot** to examine the metadata image. Apache's current example exposes `brokers`, `metadataQuorum`, `topicIds`, and `topics`, and shows partition data including topic ID, replicas, ISR, leader and epochs. Apache also warns that the offset-zero bootstrap snapshot does not contain normal cluster metadata. ¹

```
PART="$CENTRAL_EVIDENCE/$C201/kraft-metadata/__cluster_metadata-0"

ls -l "$PART"/[0-9]*-[0-9]*.checkpoint 2>/dev/null | sort
```

Then, for a selected valid snapshot:

```
kafka-metadata-shell.sh \
--snapshot \
"$PART/00000000000000722800-0000000123.checkpoint"
```

Example interactive investigation:

```
>> ls /
brokers local metadataQuorum topicIds topics

>> ls /topics

>> cat /topics/orders/0/data

>> exit
```

Do not assume that the latest snapshot alone equals the final local state. KIP-630 specifies that records at or after the snapshot end offset must subsequently be applied. ¹⁹

The forensic decision table should look like this:

Evidence	C201	C202	Interpretation
<code>cluster.id</code>	CLUSTER-A	CLUSTER-A	Must match
<code>node.id</code>	201	202	Expected
<code>directory.id</code>	UUID201	UUID202	Must remain distinct
Latest snapshot ID	0201-E201	0202-E202	Compare offset and epoch
Last local LEO	LE0201	LE0202	Local replication extent only
Latest voter set seen	...	...	Membership evidence

Evidence	C201	C202	Interpretation
Last known healthy HWM from monitoring	H	H	Minimum committed state known externally
Final controller journal epoch	...	...	Election-history evidence

Also preserve DC2 broker metadata directories before Branch B work. KIP-853 describes brokers as observers that discover the KRaft leader and consume metadata, and Kafka's status output lists observers separately from voters. A broker copy can therefore contain highly useful metadata evidence but **cannot provide the missing vote**. ²¹

For each surviving broker:

```
# Example only; replace B201/B202 with actual surviving broker hosts.
for h in b201.dc2.example.com b202.dc2.example.com; do
  ssh "$h" "
    grep -E \
      '^(node.id|metadata.log.dir|log.dirs|process.roles)= ' \
      /etc/kafka/server.properties
  "
done
```

Then snapshot or rsync the broker's metadata and data volumes before any new-quorum experiment.

Preferred recovery when a third original voter exists

Branch A applies only when an additional **original voter state** can be recovered safely.

There is an important distinction between these two cases:

GOOD CANDIDATE

An intact C101 disk,
or a crash-consistent storage copy representing its last persisted state,
including:

- meta.properties
- quorum-state
- __cluster_metadata-0 logs
- snapshots

NOT AUTOMATICALLY RPO=0-CAPABLE

A week-old backup of C101 storage
that predates unknown metadata commits.

Restoring an arbitrarily stale backup under the original `node.id` / `directory.id` is not equivalent to recovering the original current voter. Raft safety assumes persisted log/vote state survives; Apache's

failed-disk analysis explains why reintroducing a voter after persistent state loss can make committed information vulnerable. Treat old backups as forensic evidence unless their freshness and consistency are established. ²⁰

Assume here that C101 is recovered.

Recovered/available original voters:

```
C101 original node.id=101 + original directory.id
C201 original node.id=201 + original directory.id
C202 original node.id=202 + original directory.id

= 3 of original 5 voters
```

A five-voter quorum requires three. Moreover, KRaft voting considers log freshness; KIP-853 notes that a candidate still needs the appropriate longer/newer log before another replica grants its vote. This is what permits the original majority to resolve which history can safely lead rather than an operator choosing by raw LEO. ¹⁸

Validate C101 before it is started:

```
export RECOVERED=C101
export RECOVERED_HOST=c101.dc1.example.com
export C101_META=/var/lib/kafka/kraft-metadata

ssh "$RECOVERED_HOST" "
  grep -E \
    '^(cluster.id|node.id|directory.id|version)= ' \
    '${C101_META}/meta.properties'
"
```

Verify:

```
cluster.id == C201 cluster.id == C202 cluster.id
node.id == 101
directory.id == original UUID101
```

Do not use:

```
kafka-storage.sh format ...
```

on C101. It is being restored **because its persisted identity and Raft state matter**.

Determine the expected endpoint from the latest credible `VotersRecord` or a pre-disaster `describe --status`:

```
101 + UUID101 + CONTROLLER://c101.dc1.example.com:9093
```

KIP-853 stores endpoint information in `VotersRecord`, so the old endpoint should be made reachable during initial resurrection. Endpoint information can be updated dynamically later, but using DNS/NAT to preserve the recorded name is usually the least invasive emergency technique. ²

Validate from both DC2 controllers:

```
ssh "$C201" \  
  "getent hosts '$C101'; nc -vz '$C101' '$CTRL_PORT' || true"  
  
ssh "$C202" \  
  "getent hosts '$C101'; nc -vz '$C101' '$CTRL_PORT' || true"
```

The port will not answer until C101 is running, but DNS should resolve to the intended DR machine.

Fence every unrecovered copy of C101/C102/C103 before starting the recovered set. A forgotten VM or old DC1 storage instance must not later appear with the same replica identity.

Start the three original voters:

```
ssh "$C201" \  
  "sudo systemctl start '$KAFKA_SERVICE'"  
  
ssh "$C202" \  
  "sudo systemctl start '$KAFKA_SERVICE'"  
  
ssh "$C101" \  
  "sudo systemctl start '$KAFKA_SERVICE'"
```

Observe all three:

```
for h in "$C101" "$C201" "$C202"; do  
  echo "===== $h ====="  
  
  ssh "$h" "  
    sudo journalctl \  
      -u '$KAFKA_SERVICE' \  
      --since '10 minutes ago' \  
      --no-pager \  
      | grep -Ei \  
        'raft|leader|candidate|vote|epoch|quorum|fetch|snapshot|error|fatal'  
    " || true  
done
```

Now look for a leader:

```
kafka-metadata-quorum.sh \
  --command-config "$ADMIN_CONFIG" \
  --bootstrap-controller "${C201}:${CTRL_PORT}" \
  describe --status
```

A successful recovery checkpoint should show:

ClusterId:	original cluster ID
LeaderId:	101, 201, or 202
LeaderEpoch:	valid new/current epoch
HighWatermark:	numeric
CurrentVoters:	still 101,102,103,201,202 initially

The fact that C102 and C103 remain in `CurrentVoters` is expected. The quorum can operate because C101+C201+C202 provide three live voters. Current Kafka exposes those voter IDs, directory IDs and endpoints directly in `describe --status`. ¹

Immediately inspect replication:

```
kafka-metadata-quorum.sh \
  --command-config "$ADMIN_CONFIG" \
  --bootstrap-controller "${C201}:${CTRL_PORT}" \
  describe --replication
```

Do not proceed until C101/C201/C202 converge to a stable leader history and lag is zero or negligible. The new-controller procedure in KIP-853 itself requires catch-up before voter changes because changing membership while a replica is far behind can prevent high-watermark progress. ²²

Critical RPO checkpoint: compare the recovered HWM with the last known pre-disaster HWM from monitoring.

Recovered HWM >= last known healthy HWM
PASS candidate
Recovered HWM < last known healthy HWM
HARD STOP

If the recovered quorum comes up below a pre-disaster HWM that you know was committed, the selected recovery state is missing known committed metadata and must not be accepted.

Once the three-voter live subset is stable, remove the two permanently lost voters **one at a time**. KIP-853 intentionally allows only one voter change at a time and requires the changed `VotersRecord` to be committed by the appropriate majority, preserving overlapping-majority safety. ²²

First obtain the dead voter directory IDs from `describe --status`. Suppose:

```
C102 = node.id 102 / directory.id UUID102  
C103 = node.id 103 / directory.id UUID103
```

Remove C102:

```
kafka-metadata-quorum.sh \  
  --command-config "$ADMIN_CONFIG" \  
  --bootstrap-controller "${C201}:${CTRL_PORT}" \  
  remove-controller \  
  --controller-id 102 \  
  --controller-directory-id UUID102
```

Current Kafka 4.3 documents that a dynamic controller should be removed from membership with `remove-controller` and identifies the controller with both ID and directory ID. ⁵

Verify before doing anything else:

```
kafka-metadata-quorum.sh \  
  --command-config "$ADMIN_CONFIG" \  
  --bootstrap-controller "${C201}:${CTRL_PORT}" \  
  describe --status  
  
kafka-metadata-quorum.sh \  
  --command-config "$ADMIN_CONFIG" \  
  --bootstrap-controller "${C201}:${CTRL_PORT}" \  
  describe --replication
```

Membership is now conceptually:

```
101 103 201 202  
  
4 voters  
3 live  
majority = 3
```

Remove C103:

```
kafka-metadata-quorum.sh \  
  --command-config "$ADMIN_CONFIG" \  
  --bootstrap-controller "${C201}:${CTRL_PORT}" \  
  remove-controller \  
  --controller-id 103 \  
  --controller-directory-id UUID103
```

Verify again:

```
kafka-metadata-quorum.sh \
  --command-config "$ADMIN_CONFIG" \
  --bootstrap-controller "${C201}:${CTRL_PORT}" \
  describe --status
```

Desired intermediate state:

CurrentVoters:

101
201
202

3 voters
3 live
majority = 2

This is the safest point from which to restore a full five-controller topology.

Provision C203 as a **new controller joining an existing dynamic quorum**. Current Apache Kafka explicitly instructs operators to use `--no-initial-controllers` for brokers/controllers joining an existing cluster; after the controller starts as a non-voter/observer and catches up, `add-controller` makes it a voter. ¹

Example `/etc/kafka/controller-203.properties`:

```
process.roles=controller
node.id=203

controller.listener.names=CONTROLLER
listeners=CONTROLLER://c203.dc2.example.com:9093

controller.quorum.bootstrap.servers=c101.dc1.example.com:
9093,c201.dc2.example.com:9093,c202.dc2.example.com:9093

metadata.log.dir=/var/lib/kafka/kraft-metadata
```

Do not configure:

```
controller.quorum.voters=...
```

for this dynamic quorum. ¹

Use the **existing original cluster ID**:

```
export CLUSTER_ID='q1Sh-9_ISia_zwGINzRvyQ'

ssh c203.dc2.example.com "
  $KAFKA_HOME/bin/kafka-storage.sh format \
    --cluster-id '$CLUSTER_ID' \
    --config /etc/kafka/controller-203.properties \
    --no-initial-controllers
"
```

Check the newly generated storage identity:

```
ssh c203.dc2.example.com "
  grep -E \
    '^(cluster.id|node.id|directory.id|version)=' \
    /var/lib/kafka/kraft-metadata/meta.properties
"
```

Start C203:

```
ssh c203.dc2.example.com \
  "sudo systemctl start '$KAFKA_SERVICE'"
```

Monitor replication:

```
watch -n 2 \
  "kafka-metadata-quorum.sh \
    --command-config '$ADMIN_CONFIG' \
    --bootstrap-controller '${C201}:${CTRL_PORT}' \
    describe --replication"
```

Before promotion, require:

```
C203 visible as non-voting replica / observer
C203 LE0 approximately leader LE0
Lag = 0 or operationally negligible
LastFetchTimestamp recent
LastCaughtUpTimestamp recent
```

KIP-853 explicitly requires the prospective voter to catch up before `VotersRecord` is appended. ²²

Then add it. The add operation must use the **new controller's server configuration**, from which the tool obtains its replica ID, endpoint and metadata-directory identity; authentication settings required by the AdminClient may also need to be present. ⁵

Run from C203:

```
ssh c203.dc2.example.com "  
$KAFKA_HOME/bin/kafka-metadata-quorum.sh \  
--command-config /etc/kafka/controller-203.properties \  
--bootstrap-controller '${C201}:${CTRL_PORT}' \  
add-controller  
"
```

Verify:

```
kafka-metadata-quorum.sh \  
--command-config "$ADMIN_CONFIG" \  
--bootstrap-controller "${C201}:${CTRL_PORT}" \  
describe --status
```

Expected:

```
101  
201  
202  
203  
  
N = 4  
majority = 3
```

Repeat the same procedure for C204:

```
ssh c204.dc2.example.com "  
$KAFKA_HOME/bin/kafka-storage.sh format \  
--cluster-id '$CLUSTER_ID' \  
--config /etc/kafka/controller-204.properties \  
--no-initial-controllers  
  
sudo systemctl start '$KAFKA_SERVICE'  
"
```

Wait for catch-up, then:

```
ssh c204.dc2.example.com "  
$KAFKA_HOME/bin/kafka-metadata-quorum.sh \  
--command-config /etc/kafka/controller-204.properties \  
--bootstrap-controller '${C201}:${CTRL_PORT}' \  
add-controller  
"
```

Final temporary topology:

```
101 recovered
```

```
201
```

```
202
```

```
203
```

```
204
```

```
5 / 5 voters healthy
```

If C101 is temporary disaster hardware, add its permanent replacement **before** removing it. Never voluntarily reduce the live majority during reconstruction.

Branch A rollback rule: before the first successful post-disaster election, failed recovery attempts may be retried from clean working clones. Once any original controller has participated in a new election epoch or a voter change has committed, do not restore an old disk image onto one live controller independently. If a full retry becomes necessary, fence every controller, stop the entire recovery set, preserve the new attempt as evidence, and re-clone all required working storage from the immutable recovery baseline. Raft's persistent vote/log assumptions make partial time-travel of a single voter unsafe. ²⁰

Minority-only recovery and new-quorum rebuild

Branch B applies when no intact third original voter can be recovered.

At this point:

```
Authoritative voter set known from KRaft metadata:
```

```
101
```

```
102
```

```
103
```

```
201
```

```
202
```

```
Physically available original voters:
```

```
201
```

```
202
```

```
2 / 5
```

There is no majority and therefore no normal control-plane mechanism with which the minority can legitimately commit:

```
remove 101
```

```
remove 102
```

```
remove 103
```

KIP-853 membership changes are committed through `VotersRecord` updates and require majority agreement of the changed configuration. The design intentionally prevents a minority from simply redefining itself as the new quorum. ²²

Changing:

```
controller.quorum.bootstrap.servers=c201.dc2.example.com:
9093,c202.dc2.example.com:9093
```

does **not** change:

```
CurrentVoters = 101,102,103,201,202
```

because bootstrap servers are discovery addresses; voter membership is stored in KRaft log/snapshot state. ²¹

Similarly, these are not minority-recovery commands:

```
kafka-metadata-quorum.sh add-controller
kafka-metadata-quorum.sh remove-controller
```

They are normal leader-mediated membership operations. ⁵

Minority reconstruction checkpoint

Before abandoning the original quorum, build a metadata evidence dossier from:

```
C201 controller log + snapshots
C202 controller log + snapshots
all surviving broker observer metadata
pre-disaster monitoring:
    HighWatermark
    LeaderEpoch
    CurrentVoters
    CurrentObservers
configuration/IaC:
    topics
    partition counts
    replica assignments
    topic configs
    broker configs
    ACLs/security
consumer offset exports if available
transactional application inventory
MirrorMaker / remote backup / tiered-storage inventory
```

The fact that the two survivors have the same latest snapshot and identical post-snapshot records improves confidence about the DC2-visible state, but does not prove that DC1 had not committed something later. That limitation must be written into the incident decision explicitly. ⁴

Use a report such as:

```
LAST KNOWN HEALTHY HWM:
  81500342

C201:
  latest snapshot      81400000 / epoch 301
  local LE0           81500342
  latest voter set     101,102,103,201,202

C202:
  latest snapshot      81400000 / epoch 301
  local LE0           81500342
  latest voter set     101,102,103,201,202

BROKER OBSERVERS:
  metadata reaches     81500342

KNOWN HEALTHY HWM:
  81500342

ASSESSMENT:
  strong evidence that surviving state contains all metadata
  known committed at the last observation,
  but absence of later DC1-only committed metadata is unprovable.
```

This can establish a **known-good minimum**, not mathematically prove the missing maximum.

Why `--standalone` must not be run on C201/C202 originals

Current Apache Kafka documentation describes:

```
kafka-storage.sh format \
  --cluster-id <CLUSTER_ID> \
  --standalone \
  --config config/controller.properties
```

as the method for bootstrapping a **new** dynamic quorum. It creates `meta.properties` and an offset-zero bootstrap snapshot containing `KRaftVersionRecord` and a `VotersRecord` that makes that node the sole voter. ¹

KIP-853 goes further and labels `--standalone` “unsafe” in the consensus-bootstrap sense because no pre-existing quorum authorizes that initial configuration; it should be executed once for the first voter, with later voters joining through `add-controller`. ²³

Therefore this is forbidden:

```
# NEVER against original C201 survivor storage

kafka-storage.sh format \
  --cluster-id "$OLD_CLUSTER_ID" \
  --standalone \
  --config /etc/kafka/controller.properties
```

It is not “turn existing C201 history into a one-node quorum.” It formats storage and creates a new bootstrap state. ⁵

Likewise, `--initial-controllers` initializes a brand-new voter set in bootstrap snapshots. It is not a quorum-loss recovery override for an existing metadata history. ⁵

The supported operational boundary

After reviewing Apache Kafka 4.3 operations documentation, KIP-853, provisioning commands and failed-metadata-disk guidance, I found **no documented stock Apache Kafka operator workflow that imports an existing 2/5 minority log into a newly authoritative quorum while preserving that log as the official history**. The documented workflows are new-quorum bootstrap, new-node provisioning, and leader-mediated dynamic membership changes. ²⁴

That gives Branch B two realistic choices:

```
B1  Metadata-preserving offline salvage
-----
Keep C201/C202 and broker copies immutable.
Work with Apache Kafka engineering / distribution vendor.
Reconstruct a trusted metadata image offline.
Validate it in an isolated laboratory.
Do not improvise byte-level VotersRecord edits in production.

B2  New-quorum / new-cluster rebuild
-----
Build a clean KRaft quorum on fresh storage.
Recreate control-plane objects.
Restore/replay business data from an independent source.
```

If raw broker segments are the **only** surviving copy of the business records, Branch B1 should remain open while those disks are preserved. Do not begin destructive reformatting merely to reduce RTO.

Building a genuinely new dynamic quorum

The safest production separation is to use new controller IDs, new storage paths and preferably a **new cluster ID**. Reusing the old cluster ID for an empty metadata plane can make it deceptively similar to the destroyed cluster while it lacks the old topics, topic IDs, assignments, ACL/config metadata and

coordinator state. That is operationally dangerous; use the old ID only in a vendor-supported salvage plan.

Example new controllers:

```
C301
C302
C303
```

Example `/etc/kafka-rebuild/controller-301.properties`:

```
process.roles=controller
node.id=301

controller.listener.names=CONTROLLER
listeners=CONTROLLER://c301.dc2.example.com:9093

controller.quorum.bootstrap.servers=c301.dc2.example.com:
9093,c302.dc2.example.com:9093,c303.dc2.example.com:9093

metadata.log.dir=/var/lib/kafka-new/kraft-metadata
```

Create the new cluster ID:

```
export NEW_CLUSTER_ID="$(
  kafka-storage.sh random-uuid
)"

printf 'NEW_CLUSTER_ID=%s\n' "$NEW_CLUSTER_ID"
```

Apache requires the same generated cluster ID when formatting nodes belonging to the new cluster.

1

Bootstrap **C301 only**:

```
ssh c301.dc2.example.com "
  $KAFKA_HOME/bin/kafka-storage.sh format \
    --cluster-id '$NEW_CLUSTER_ID' \
    --standalone \
    --config /etc/kafka-rebuild/controller-301.properties
"
```

Confirm:

```
ssh c301.dc2.example.com "  
  grep -E \  
    '^(cluster.id|node.id|directory.id|version)= ' \  
    /var/lib/kafka-new/kraft-metadata/meta.properties  
"
```

Start it:

```
ssh c301.dc2.example.com \  
  "sudo systemctl start kafka-controller-rebuild"
```

Verify the newly bootstrapped one-voter quorum:

```
kafka-metadata-quorum.sh \  
  --command-config /etc/kafka-rebuild/admin.properties \  
  --bootstrap-controller c301.dc2.example.com:9093 \  
  describe --status
```

Expected initial membership:

```
CurrentVoters = [301]
```

This is precisely what current Kafka documents `--standalone` as creating. ¹

Prepare C302:

```
ssh c302.dc2.example.com "  
  $KAFKA_HOME/bin/kafka-storage.sh format \  
    --cluster-id '$NEW_CLUSTER_ID' \  
    --config /etc/kafka-rebuild/controller-302.properties \  
    --no-initial-controllers  
  
  sudo systemctl start kafka-controller-rebuild  
"
```

Monitor from C301:

```
kafka-metadata-quorum.sh \  
  --command-config /etc/kafka-rebuild/admin.properties \  
  --bootstrap-controller c301.dc2.example.com:9093 \  
  describe --replication
```

When C302 catches up:

```
ssh c302.dc2.example.com "  
$KAFKA_HOME/bin/kafka-metadata-quorum.sh \  
--command-config /etc/kafka-rebuild/controller-302.properties \  
--bootstrap-controller c301.dc2.example.com:9093 \  
add-controller  
"
```

Verify:

```
kafka-metadata-quorum.sh \  
--command-config /etc/kafka-rebuild/admin.properties \  
--bootstrap-controller c301.dc2.example.com:9093 \  
describe --status
```

Expected:

```
CurrentVoters = 301,302
```

Now provision C303:

```
ssh c303.dc2.example.com "  
$KAFKA_HOME/bin/kafka-storage.sh format \  
--cluster-id '$NEW_CLUSTER_ID' \  
--config /etc/kafka-rebuild/controller-303.properties \  
--no-initial-controllers  
  
sudo systemctl start kafka-controller-rebuild  
"
```

Wait for catch-up:

```
kafka-metadata-quorum.sh \  
--command-config /etc/kafka-rebuild/admin.properties \  
--bootstrap-controller c301.dc2.example.com:9093 \  
describe --replication
```

Then:

```
ssh c303.dc2.example.com "  
$KAFKA_HOME/bin/kafka-metadata-quorum.sh \  
--command-config /etc/kafka-rebuild/controller-303.properties \  
--bootstrap-controller c301.dc2.example.com:9093 \  
"
```

```
add-controller  
"
```

Final check:

```
kafka-metadata-quorum.sh \  
  --command-config /etc/kafka-rebuild/admin.properties \  
  --bootstrap-controller c301.dc2.example.com:9093 \  
  describe --status
```

Desired:

```
LeaderId:      301/302/303  
HighWatermark: advancing/stable  
CurrentVoters: 301,302,303
```

Then verify feature level:

```
kafka-features.sh \  
  --command-config /etc/kafka-rebuild/admin.properties \  
  --bootstrap-controller c301.dc2.example.com:9093 \  
  describe
```

A dynamic quorum is represented by `kraft.version >= 1` in the current operator documentation.

1

At this point you have a healthy new KRaft control plane, but you have not recovered the old cluster. That distinction is crucial.

Broker-data recovery, rollback, and validation

KRaft control-plane loss and broker data loss are different events. The old brokers may still hold physical log segments for user topics and Kafka internal topics even while the KRaft metadata quorum is unavailable. However, KRaft metadata describes the topics themselves: topic IDs, partitions, replicas, ISR, leaders, configuration and other cluster state. The metadata shell example in the Apache documentation shows exactly this mapping. ¹

Therefore:

```
surviving broker *.log files  
  !=  
self-describing replacement KRaft control plane
```

A clean new quorum does not automatically scan arbitrary broker `log.dirs` and reconstruct all old KRaft topic metadata. I found no documented Apache Kafka 4.3 CLI that performs such an import. The

safe runbook must therefore treat old broker disks as **recovery sources**, not immediately attach them to an empty new control plane. 25

Before Branch B cutover, preserve every surviving broker volume:

```
ssh b201.dc2.example.com "  
  sudo systemctl stop kafka  
  
  sudo rsync \  
    -aHAX \  
    --numeric-ids \  
    /var/lib/kafka/data/ \  
    '/recovery/${INCIDENT_ID}/broker-data/'  
"
```

Hash it:

```
ssh b201.dc2.example.com "  
  (  
    cd '/recovery/${INCIDENT_ID}/broker-data'  
    find . -type f -print0 \  
      | LC_ALL=C sort -z \  
      | xargs -0 sha256sum  
  ) > '/recovery/${INCIDENT_ID}/broker-data.sha256'  
"
```

Repeat for every surviving broker.

Reconstructing ordinary topic metadata

From decoded controller snapshots/logs and metadata-shell evidence, record at least:

```
Topic name  
Old TopicId  
Partition count  
Per-partition replicas  
ISR at last credible metadata state  
Leader  
Topic-level configs  
min.insync.replicas  
cleanup.policy  
retention settings  
compression / message-size overrides
```

For example, a metadata-shell object may show:

```
{
  "partitionId": 0,
  "topicId": "5zoAlv-xEh9xRANKXt1Lbg",
  "replicas": [201, 202, 101, 102],
  "isr": [201, 202, 101, 102],
  "leader": 201,
  "leaderEpoch": 45,
  "partitionEpoch": 17
}
```

Apache's metadata shell exposes this type of partition information from KRaft snapshots. ¹

In a genuinely new cluster, recreate user topics before restoring/replaying their business data:

```
kafka-topics.sh \
  --bootstrap-server new-kafka.example.com:9092 \
  --create \
  --topic orders \
  --partitions 48 \
  --replication-factor 3
```

Then restore topic configuration:

```
kafka-configs.sh \
  --bootstrap-server new-kafka.example.com:9092 \
  --entity-type topics \
  --entity-name orders \
  --alter \
  --add-config \
  'cleanup.policy=delete,min.insync.replicas=2,retention.ms=604800000'
```

Current Apache documentation supports topic creation/configuration this way. ¹⁰

Do not copy raw old partition directories directly into newly created topic directories as a routine production procedure. Newly created topics have newly established control-plane identity, and Apache Kafka does not document raw segment-directory transplantation as a supported new-cluster migration mechanism.

Where business data exists in independent durable form—MirrorMaker/cluster replication, object storage, application source-of-truth, backup export or another Kafka cluster—replay from that supported source into the new cluster. If only raw Kafka segments remain, keep them immutable and move the incident into specialized data-salvage engineering rather than guessing.

`__consumer_offsets`

Consumer-group committed offsets are themselves Kafka records stored in the compacted internal topic `__consumer_offsets`. The group coordinator appends offset commits there and reloads group offset state from that topic. ¹¹

In Branch A, where the original cluster metadata/control plane is genuinely restored, surviving replicas of `__consumer_offsets` can continue to represent the original group state subject to ordinary partition recovery.

In Branch B, the new cluster's `__consumer_offsets` is a new coordinator state. **Do not assume old consumer group positions reappear because business topic logs survived.**

If external offset exports exist, restore them only after the target topics and their record ranges are validated and all members of the consumer group are stopped. Apache documents `--reset-offsets`, `--execute`, `--export` and `--from-file`, and explicitly requires consumers to be inactive during offset reset. ²⁶

Dry-run an offset restore first:

```
kafka-consumer-groups.sh \  
  --bootstrap-server new-kafka.example.com:9092 \  
  --reset-offsets \  
  --group payments-consumer \  
  --from-file payments-consumer-offsets.csv
```

Then, after verifying the proposed offsets:

```
kafka-consumer-groups.sh \  
  --bootstrap-server new-kafka.example.com:9092 \  
  --reset-offsets \  
  --group payments-consumer \  
  --from-file payments-consumer-offsets.csv \  
  --execute
```

Apache notes that out-of-range reset targets are adjusted to the available end offset, which is another reason to compare the proposed values with the restored topic's earliest and latest offsets before executing. ²⁷

Do not manually alter the partition count of `__consumer_offsets`. Current Kafka operations documentation explicitly warns operators not to manually increase partitions for internal state topics such as `__consumer_offsets`, `__transaction_state`, `__share_group_state` and `__cluster_metadata`, because coordinator mappings and state can be corrupted. ¹⁰

`__transaction_state`

Kafka's transaction coordinator uses its internal transaction state topic; current broker configuration exposes `transaction.state.log.replication.factor`, `transaction.state.log.min.isr`,

partition count and other transaction-log controls. Kafka 4.3 defaults include transaction-state replication factor 3 and minimum ISR 2; internal topic creation waits until the cluster can satisfy its configured replication factor. ¹²

Branch A can potentially retain the original transactional control state because the original cluster/control plane and surviving internal-topic replicas are being recovered.

Branch B cannot safely claim seamless continuity of in-flight transactions merely by rebuilding ordinary topics. Transactional producers must be restarted in a controlled fashion after data restoration; business owners should explicitly account for transactions that were in progress at the disaster boundary. Do not manually manipulate the internal transaction-topic partition topology. ²⁸

For exactly-once applications, the cutover acceptance criteria should include application-level duplicate/loss reconciliation around the disaster boundary rather than relying solely on “Kafka is green.”

Security and control-plane metadata

A KRaft metadata image also contains more than topics: dynamic configuration and other controller-managed cluster state must be reconstructed from the recovered metadata evidence or infrastructure-as-code before clients are admitted. Current broker documentation explicitly notes that dynamic broker and cluster defaults are stored in metadata state, so a clean quorum does not inherit those settings from old properties files automatically. ¹²

At minimum compare:

```
broker configs
topic configs
ACL policy
listener/security configuration
SCRAM or other credentials where applicable
quotas
feature versions
broker registrations
topic/partition map
consumer offsets
transactional application inventory
```

Branch B rollback

The safest feature of a proper Branch B rebuild is that the old cluster remains untouched.

Use:

```
OLD:
C201 old metadata disk      READ-ONLY / OFFLINE
C202 old metadata disk      READ-ONLY / OFFLINE
old broker disks            READ-ONLY / OFFLINE
```

NEW:

- new cluster ID
- C301/C302/C303
- new controller storage
- new/fresh broker storage

If a valid C101/C102/C103 disk is later discovered, stop the new-cluster migration before destructive data transformations and reassess Branch A. The original C201/C202 state is still available because the new quorum never reformatted it.

Post-recovery validation

Whether Branch A or Branch B is used, do not restore client traffic merely because `LeaderId` is non-negative.

First check quorum:

```
kafka-metadata-quorum.sh \  
  --command-config "$ADMIN_CONFIG" \  
  --bootstrap-controller "$ACTIVE_CONTROLLER" \  
  describe --status  
  
kafka-metadata-quorum.sh \  
  --command-config "$ADMIN_CONFIG" \  
  --bootstrap-controller "$ACTIVE_CONTROLLER" \  
  describe --replication
```

Require:

- stable `LeaderId`
- stable/increasing `LeaderEpoch` only when elections occur
- `HighWatermark` available and advancing when metadata writes occur
- expected `CurrentVoters`
- correct `directoryId` for every voter
- no unexpected stale voters
- low/zero voter lag
- recent catch-up timestamps

The relevant fields are directly defined by current Kafka's quorum tools. ²⁹

Verify dynamic KRaft feature level:

```
kafka-features.sh \  
  --bootstrap-controller "$ACTIVE_CONTROLLER" \  
  --command-config "$ADMIN_CONFIG" \  
  describe
```

Check broker registration:

```
kafka-broker-api-versions.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP"
```

List and describe topics:

```
kafka-topics.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --list  
  
kafka-topics.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --describe
```

Check under-replicated partitions:

```
kafka-topics.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --describe \  
  --under-replicated-partitions
```

Check broker log directories:

```
kafka-log-dirs.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --describe
```

Check consumer groups:

```
kafka-consumer-groups.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --list
```

For a critical group:

```
kafka-consumer-groups.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --describe \  
  --group payments-consumer
```

Kafka's current operations documentation uses these consumer-group descriptions to expose current offset, log end offset and lag. ¹⁰

Create a dedicated recovery canary topic rather than testing with business topics:

```
kafka-topics.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --create \  
  --topic __dr_recovery_canary \  
  --partitions 3 \  
  --replication-factor 3
```

Produce:

```
printf '%s\n' \  
  "incident=${INCIDENT_ID} timestamp=$(date -u --iso-8601=seconds)" \  
| kafka-console-producer.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --topic __dr_recovery_canary
```

Consume:

```
kafka-console-consumer.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --topic __dr_recovery_canary \  
  --from-beginning \  
  --max-messages 1
```

Delete the canary after validation if policy permits:

```
kafka-topics.sh \  
  --bootstrap-server "$BROKER_BOOTSTRAP" \  
  --delete \  
  --topic __dr_recovery_canary
```

Final production acceptance should require three independent sign-offs:

Layer	Required evidence
KRaft control plane	Expected voters, stable leader, valid HWM, voter lag healthy
Kafka data plane	Expected brokers/topics/partitions, replicas/ISR healthy, data-range checks pass
Application state	Consumer offsets validated, transactional applications reconciled, canary and business smoke tests pass

Operator scripts, evidence template, and action flow

The following collection script can be stored in the runbook repository and executed **only after the local Kafka controller process has been stopped**. It deliberately refuses to call `kafka-storage` format, alter files, or change membership.

```
#!/usr/bin/env bash
#
# collect-kraft-controller-evidence.sh
#
# Run locally on a STOPPED Kafka controller.
#
# Usage:
#   sudo ./collect-kraft-controller-evidence.sh \
#       dc1-loss-20260902T101500Z \
#       /var/lib/kafka/kraft-metadata \
#       /recovery
#

set -Eeuo pipefail
umask 077

if [[ $# -ne 3 ]]; then
    echo "Usage: $0 <incident-id> <metadata-dir> <evidence-root>" >&2
    exit 64
fi

INCIDENT_ID="$1"
META_DIR="$2"
EVIDENCE_ROOT="$3"

HOST="$(hostname -f 2>/dev/null || hostname)"
OUT="${EVIDENCE_ROOT}/${INCIDENT_ID}/${HOST}"
COPY="${OUT}/kraft-metadata"
PART="${COPY}/__cluster_metadata-0"

mkdir -p "$OUT"

echo "[INFO] Incident: ${INCIDENT_ID}"
echo "[INFO] Host:     ${HOST}"
echo "[INFO] Source:    ${META_DIR}"
echo "[INFO] Output:    ${OUT}"

if pgrep -af 'kafka.Kafka|KafkaRaftServer' >/dev/null 2>&1; then
    echo "[FATAL] Kafka appears to be running." >&2
    echo "[FATAL] Stop the controller before forensic collection." >&2
    pgrep -af 'kafka.Kafka|KafkaRaftServer' >&2 || true
    exit 70
fi
```

```

if [[ ! -f "${META_DIR}/meta.properties" ]]; then
    echo "[FATAL] ${META_DIR}/meta.properties not found." >&2
    exit 66
fi

if [[ ! -d "${META_DIR}/__cluster_metadata-0" ]]; then
    echo "[FATAL] __cluster_metadata-0 not found." >&2
    exit 66
fi

echo "[INFO] Recording host information..."

{
    echo "incident_id=${INCIDENT_ID}"
    echo "hostname=${HOST}"
    echo "timestamp_utc=$(date -u --iso-8601=seconds)"
    echo
    uname -a
    echo
    timedatectl 2>&1 || true
    echo
    df -hT "${META_DIR}"
    echo
    mount | grep -F "${META_DIR}" || true
} > "${OUT}/host-info.txt"

echo "[INFO] Copying metadata..."

mkdir -p "${COPY}"

rsync \
    -aHAX \
    --numeric-ids \
    "${META_DIR}/" \
    "${COPY}/"

echo "[INFO] Recording identity..."

grep -E \
    '^(cluster.id|node.id|directory.id|version)= ' \
    "${COPY}/meta.properties" \
    > "${OUT}/identity.txt"

echo "[INFO] Inventorying files..."

find "${COPY}" \
    -type f \
    -printf '%P\t%s\t%TY-%Tm-%TdT%TH:%TM:%TS\n' \
    | LC_ALL=C sort \
    > "${OUT}/files.tsv"

```

```

echo "[INFO] Hashing every copied file..."

(
  cd "$COPY"

  find . \
    -type f \
    -print0 \
    | LC_ALL=C sort -z \
    | xargs -0 sha256sum
) > "${OUT}/files.sha256"

if [[ -f "${PART}/quorum-state" ]]; then
  sha256sum \
    "${PART}/quorum-state" \
    > "${OUT}/quorum-state.sha256"

  stat \
    "${PART}/quorum-state" \
    > "${OUT}/quorum-state.stat"
fi

echo "[INFO] Creating evidence archive..."

tar \
  --xattrs \
  --acls \
  --numeric-owner \
  -C "$OUT" \
  -cpf "${OUT}/${INCIDENT_ID}-${HOST}.tar" \
  kraft-metadata \
  identity.txt \
  files.tsv \
  files.sha256 \
  host-info.txt

sha256sum \
  "${OUT}/${INCIDENT_ID}-${HOST}.tar" \
  > "${OUT}/${INCIDENT_ID}-${HOST}.tar.sha256"

echo
echo "[OK] Evidence collection complete."
echo "[OK] Archive:"
echo "    ${OUT}/${INCIDENT_ID}-${HOST}.tar"
echo "[OK] SHA256:"
cat "${OUT}/${INCIDENT_ID}-${HOST}.tar.sha256"

```

A separate decoding script should operate on copies only:

```

#!/usr/bin/env bash
#
# decode-kraft-evidence.sh
#

set -Eeuo pipefail

if [[ $# -ne 2 ]]; then
    echo "Usage: $0 <copied-metadata-dir> <output-dir>" >&2
    exit 64
fi

META_COPY="$1"
OUT="$2"
PART="${META_COPY}/__cluster_metadata-0"

: "${KAFKA_HOME:=/opt/kafka}"

DUMP="${KAFKA_HOME}/bin/kafka-dump-log.sh"

mkdir -p "$OUT"

if [[ ! -x "$DUMP" ]]; then
    echo "kafka-dump-log.sh not found: $DUMP" >&2
    exit 66
fi

echo "[INFO] Decoding log segments..."

for f in "$PART"/*.log; do
    [[ -e "$f" ]] || continue

    base="$(basename "$f")"

    "$DUMP" \
        --cluster-metadata-decoder \
        --files "$f" \
        > "$OUT/${base}.decoded.txt" \
        2>&1
done

echo "[INFO] Decoding snapshots..."

while IFS= read -r -d '' f; do
    base="$(basename "$f")"

    "$DUMP" \
        --cluster-metadata-decoder \
        --files "$f" \
        > "$OUT/${base}.decoded.txt" \

```

```

2>&1

done < <(
  find "$PART" \
    -maxdepth 1 \
    -type f \
    -name '[0-9]*-[0-9]*.checkpoint' \
    -print0
)

echo "[INFO] Building record index..."

grep -RniE \
  'VotersRecord|KRaftVersionRecord|TopicRecord|PartitionRecord|ConfigRecord' \
  "$OUT" \
  > "$OUT/record-index.txt" \
  || true

echo "[OK] Decode complete: $OUT"

```

Apache officially documents the `kafka-dump-log.sh --cluster-metadata-decoder` operation used by this script for both metadata log segments and snapshots. ¹

The incident evidence report should be filled in before any destructive Branch B action:

INCIDENT

=====

Incident ID:
 UTC failure time:
 Incident commander:
 Kafka exact version:
 metadata.version:
 kraft.version:
 Original ClusterId:

ORIGINAL QUORUM

=====

Controller	NodeId	DirectoryId	Endpoint
C101	101	_____	_____
C102	102	_____	_____
C103	103	_____	_____
C201	201	_____	_____
C202	202	_____	_____

PRE-INCIDENT EVIDENCE

=====

Last known LeaderId:
Last known LeaderEpoch:
Last known HighWatermark:
Timestamp of observation:
Monitoring source:

C201 FORENSICS

=====

ClusterId:
NodeId:
DirectoryId:
Latest snapshot offset:
Latest snapshot epoch:
Local log end:
Latest visible VotersRecord:
Last controller-log epoch:
SHA256 manifest:

C202 FORENSICS

=====

ClusterId:
NodeId:
DirectoryId:
Latest snapshot offset:
Latest snapshot epoch:
Local log end:
Latest visible VotersRecord:
Last controller-log epoch:
SHA256 manifest:

BROKER OBSERVER EVIDENCE

=====

Broker:
Metadata snapshot:
Metadata offset:
User-topic data volumes preserved:
__consumer_offsets replicas preserved:
__transaction_state replicas preserved:

DECISION

=====

Third original voter found: YES / NO

If YES:

voter:
NodeId:
DirectoryId:
storage freshness evidence:
endpoint restored:
Branch A approved by:

If NO:

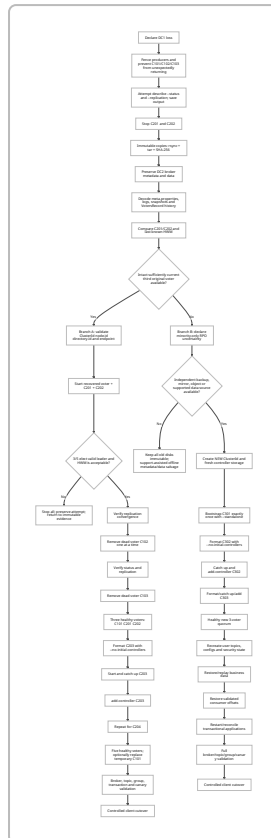
inability to prove final committed metadata acknowledged: YES / NO
independent data restore source:
new ClusterId:
Branch B approved by:

POST-RECOVERY

=====

LeaderId:
LeaderEpoch:
HighWatermark:
CurrentVoters:
Max follower lag:
Brokers registered:
Topics validated:
Under-replicated partitions:
Consumer groups validated:
Transactional apps validated:
Canary passed:
Business reconciliation passed:
Cutover UTC:

The complete action flow should be treated as two separate recovery modes, not as one continuous sequence:



The timeline can be summarized operationally as:

T0	DC1 declared unrecoverable
	NO FORMAT / NO MEMBERSHIP CHANGE
v	
T+	C201/C202 frozen
v	
T+	Controller + broker evidence hashed
v	
T+	Snapshot/log/VotersRecord comparison
+-----+	
third original voter	no third original voter
v	v
restore original 3/5	declare RP0 uncertainty
v	
Raft elects leader	v
	new isolated cluster
v	
remove lost voters	v

```

one at a time          --standalone ONCE
|                      |
v                      v
--no-initial-controllers  add C302/C303
for replacements        |
|                      v
v                      recreate metadata
add-controller          |
|                      v
v                      restore/replay data
5 healthy voters        |
|                      v
+-----> validation <---+
|
v
application cutover

```

The final operator rule is:

```

When an original majority can be reconstructed:
    let Raft recover Raft.

When only a minority survives:
    preserve it as evidence;
    do not pretend it is a majority.

--no-initial-controllers
    means "join an existing dynamic quorum."

--standalone
    means "bootstrap a new quorum on formatted storage."

controller.quorum.bootstrap.servers
    means "where can I find quorum?"

VotersRecord
    means "who is currently in quorum?"

meta.properties
    means "which cluster/node/storage incarnation am I?"

quorum-state
    means "what persistent Raft election state do I have?"

largest LEO
    does not mean "last committed truth."

```

Those distinctions are exactly why KIP-853 added persisted voter identities, dynamic `VotersRecord` state and explicit quorum-change protocols rather than allowing configuration files or minority replicas to redefine consensus locally. ³⁰

1 5 18 24 25 29 **KRaft | Apache Kafka**

<https://kafka.apache.org/43/operations/kraft/>

2 3 4 7 17 21 22 23 30 **KIP-853: KRaft Controller Membership Changes - Apache Kafka - Apache Software Foundation**

[https://cwiki.apache.org/confluence/spaces/KAFKA/pages/217391519/](https://cwiki.apache.org/confluence/spaces/KAFKA/pages/217391519/KIP-853%2BKraft%2BController%2BMembership%2BChanges)

[KIP-853%2BKraft%2BController%2BMembership%2BChanges](https://cwiki.apache.org/confluence/spaces/KAFKA/pages/217391519/KIP-853%2BKraft%2BController%2BMembership%2BChanges)

6 13 **Quick Start | Apache Kafka**

https://kafka.apache.org/43/getting-started/quickstart?utm_source=chatgpt.com

8 19 **KIP-630: Kafka Raft Snapshot - Apache Kafka - Apache Software Foundation**

<https://cwiki.apache.org/confluence/spaces/KAFKA/pages/158864763/KIP-630%2BKafka%2BRaft%2BSnapshot>

9 14 15 16 **Hardware and OS | Apache Kafka**

<https://kafka.apache.org/43/operations/hardware-and-os/>

10 28 **Basic Kafka Operations | Apache Kafka**

https://kafka.apache.org/43/operations/basic-kafka-operations/?utm_source=chatgpt.com

11 **Distribution | Apache Kafka**

https://kafka.apache.org/43/implementation/distribution/?utm_source=chatgpt.com

12 **Broker Configs | Apache Kafka**

https://kafka.apache.org/43/configuration/broker-configs/?utm_source=chatgpt.com

20 **[KAFKA-14077] KRaft should support recovery from failed disk - ASF Jira**

https://issues.apache.org/jira/browse/KAFKA-14077?utm_source=chatgpt.com

26 27 **Basic Kafka Operations | Apache Kafka**

<https://kafka.apache.org/43/operations/basic-kafka-operations/>